

Iniziato	lunedì, 8 giugno 2026, 09:01
Stato	Completato
Terminato	lunedì, 8 giugno 2026, 10:35
Tempo impiegato	1 ora 34 min.
Valutazione	15,75 su un massimo di 18,00 (87,5%)

Informazione

[ARES simulator](#) / [Simulatore ARES](#)

Domanda 1

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Di seguito viene riportata una sequenza di nove richieste di dati a una memoria cache a **mappatura diretta** di otto blocchi, inizialmente vuota. Gli **indirizzi dei blocchi** nelle richieste sono espressi in **binario a 5 bit**.

Numero della richiesta - Indirizzo in binario:

1. 00000
2. 11110
3. 10000
4. 10110
5. 11110
6. 11000
7. 10000
8. 10110
9. 11110

[0.5 punti] Completare la tabella seguente riportando lo stato della cache **dopo le nove richieste**, indicando per ogni blocco il **tag** del blocco memorizzato, o se il blocco è ancora **non valido**.

• Linea/line: 000	Tag:	10	✓
• Linea/line: 001	Tag:	Non valido/Invalid	✓
• Linea/line: 010	Tag:	Non valido/Invalid	✓
• Linea/line: 011	Tag:	Non valido/Invalid	✓
• Linea/line: 100	Tag:	Non valido/Invalid	✓
• Linea/line: 101	Tag:	Non valido/Invalid	✓
• Linea/line: 110	Tag:	11	✓
• Linea/line: 111	Tag:	Non valido/Invalid	✓

[0.5 punti] Quanti sono stati i **hit** e i **miss** nella cache?

- Hit: ✓
- Miss: ✓

Domanda 2

Parzialmente corretta

Punteggio ottenuto 0,75 su 1,00

Determinare se sono vere o false le seguenti affermazioni:

- Nella macchina di Von Neumann dati e programma sono rappresentati entrambi come sequenze di bit in memoria centrale.
 ✓
- Nell'architettura di Von Neumann, i programmi sono circuiti combinatori, composti da porte logiche dotate di 1 o più ingressi. ✓
- Il livello ISA (Instruction Set Architecture) è il livello del linguaggio macchina che descrive le istruzioni eseguite dai circuiti elettronici. ✓
- Nella CPU di Von Neumann, un ciclo di data path consiste nel processo di far passare due operandi attraverso l'ALU e memorizzarne il risultato. ✗

Domanda 3

Parzialmente corretta

Punteggio ottenuto 0,50 su 1,00

Si consideri che la memoria principale contenga i valori riportati nella figura, dove il numero a sinistra della cella rappresenta l'indirizzo della cella.

Memoria

7	0x87
6	0x86
5	0x85
4	0x84
3	0x83
2	0x82
1	0x81
0	0x80

- Se il contenuto memorizzato dal registro **x4** fosse **0xFFFFFFFF** e se quello del registro **x5** fosse **0x00000000** quale sarebbe il valore (4 byte in esadecimale) del registro **x4** se il processore eseguisse l'istruzione **lhu x4, 2(x5)**?



- Se il contenuto memorizzato dal registro **x4** fosse **0x00112233** e se quello del registro **x5** fosse **0x00000000** quale sarebbe il valore del byte all'indirizzo 3 se il processore eseguisse l'istruzione **sh x4, 0(x5)**?



Riportare i risultati in esadecimale, senza alcun carattere di spaziatura (ad esempio: 0xFFFFFFFF o 0xFF).

Domanda 4

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Si considerino due diverse implementazioni dello stesso set di istruzioni che può essere suddiviso in quattro classi: A, B, C e D, basate sui loro CPI (Cicli Per Istruzione).

Il processore P1 ha una frequenza di clock di 4 GHz e CPI di 10 (istruzioni di classe A), 5 (istruzioni di classe B), 1 (istruzioni di classe C) e 10 (istruzioni di classe D).

Il processore P2 ha una frequenza di clock di 2 GHz e CPI di 10 (istruzioni di classe A), 1 (istruzioni di classe B), 1 (istruzioni di classe C) e 1 (istruzioni di classe D).

Si consideri un programma costituito da 10000 istruzioni così suddivise: 20% di classe A, 30% di classe B, 30% di classe C e 20% di classe D.

- [0.3 punti] Determinare il numero di cicli di clock richiesti da P1 per eseguire il programma. ✓
- [0.3 punti] Determinare il numero di cicli di clock richiesti da P2 per eseguire il programma. ✓
- [0.4 punti] Quale processore esegue il programma più velocemente? ✓

Domanda 5

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Considera la tabella con i formati di istruzione RISC-V mostrata nella figura seguente. Determina il tipo di istruzione e l'istruzione equivalente in linguaggio assembly per la seguente istruzione:

0000 0000 0010 0001 0000 0001 1001 0011

- (0,50 punti) Tipo di istruzione: ✓
- (0,50 punti) Istruzione assembly: ✓

RISC-V Instruction Set

Core Instruction Formats

31	25	24	20	19	15	14	12	11	7	6	0	
funct7		rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]				rs1		funct3		rd		opcode		I-type
imm[11:5]		rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]		rs2		rs1		funct3		imm[4:1 11]		opcode		SB-type
		imm[31:12]						rd		opcode		U-type
		imm[20 10:1 11 19:12]						rd		opcode		UJ-type

RV32I Base Integer Instructions

Inst	Name	FMT	Opcode	funct3	funct7	Description (C)	Note
add	ADD	R	0110011	0x0	0x00	rd = rs1 + rs2	
sub	SUB	R	0110011	0x0	0x20	rd = rs1 - rs2	
xor	XOR	R	0110011	0x4	0x00	rd = rs1 ^ rs2	
or	OR	R	0110011	0x6	0x00	rd = rs1 rs2	
and	AND	R	0110011	0x7	0x00	rd = rs1 & rs2	
sll	Shift Left Logical	R	0110011	0x1	0x00	rd = rs1 << rs2	
srl	Shift Right Logical	R	0110011	0x5	0x00	rd = rs1 >> rs2	
sra	Shift Right Arith	R	0110011	0x5	0x20	rd = rs1 >> rs2	msb-ext
slt	Set Less Than	R	0110011	0x2	0x00	rd = (rs1 < rs2)?1:0	signed
sltu	Set Less Than (U)	R	0110011	0x3	0x00	rd = (rs1 < rs2)?1:0	unsigned
addi	ADD (Immediate)	I	0010011	0x0		rd = rs1 + imm	
xori	XOR (Immediate)	I	0010011	0x4		rd = rs1 ^ imm	
ori	OR (Immediate)	I	0010011	0x6		rd = rs1 imm	
andi	AND (Immediate)	I	0010011	0x7		rd = rs1 & imm	
slli	Shift Left Logical Imm	I	0010011	0x1	imm[11:5]=0x00	rd = rs1 << imm[0:4]	zero-ext
srl	Shift Right Logical Imm	I	0010011	0x5	imm[11:5]=0x00	rd = rs1 >> imm[0:4]	msb-ext
srai	Shift Right Arith Imm	I	0010011	0x5	imm[11:5]=0x20	rd = rs1 >> imm[0:4]	signed
slti	Set Less Than Imm	I	0010011	0x2		rd = (rs1 < imm)?1:0	unsigned
sltiu	Set Less Than Imm (U)	I	0010011	0x3		rd = (rs1 < imm)?1:0	
lb	Load Byte	I	0000011	0x0		rd = M[rs1+imm][0:7]	
lh	Load Half	I	0000011	0x1		rd = M[rs1+imm][0:15]	
lw	Load Word	I	0000011	0x2		rd = M[rs1+imm][0:31]	
lbu	Load Byte (U)	I	0000011	0x4		rd = M[rs1+imm][0:7]	zero-ext
lhu	Load Half (U)	I	0000011	0x5		rd = M[rs1+imm][0:15]	zero-ext
sb	Store Byte	S	0100011	0x0		M[rs1+imm][0:7] = rs2[0:7]	
sh	Store Half	S	0100011	0x1		M[rs1+imm][0:15] = rs2[0:15]	
sw	Store Word	S	0100011	0x2		M[rs1+imm][0:31] = rs2[0:31]	
beq	Branch ==	SB	1100011	0x0		if(rs1 == rs2) PC += imm	
bne	Branch !=	SB	1100011	0x1		if(rs1 != rs2) PC += imm	
blt	Branch <	SB	1100011	0x4		if(rs1 < rs2) PC += imm	signed
bge	Branch ≥	SB	1100011	0x5		if(rs1 ≥ rs2) PC += imm	signed
bltu	Branch < (U)	SB	1100011	0x6		if(rs1 < rs2) PC += imm	unsigned
bgeu	Branch ≥ (U)	SB	1100011	0x7		if(rs1 ≥ rs2) PC += imm	unsigned
jal	Jump And Link	UJ	1101111			rd = PC+4; PC += imm	
jalr	Jump And Link Reg	I	1100111	0x0		rd = PC+4; PC = rs1 + imm	
lui	Load Upper Imm	U	0110111			rd = imm << 12	
auipc	Add Upper Imm to PC	U	0010111			rd = PC + (imm << 12)	
ecall	Environment Call	I	1110011	0x0	imm=0x0	Context switch to OS	
ebreak	Environment Break	I	1110011	0x0	imm=0x1	Context switch to debugger	
fence	Memory fence	FENCE	0001111	0x0		Order mem/io accesses	

Domanda 6

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Due compilatori diversi hanno tradotto il seguente frammento di codice C:

```
if (!(a && b)) {  
    x = 1;  
}
```

Assumi che **a** sia in **t0**, **b** sia in **t1**, e **x** sia in **t2**.

Compilatore A:

```
1      and t3, t0, t1  
2      beqz t3, xset  
3      j skip  
4 xset: li t2, 1  
5 skip: nop
```

Compilatore B (incompleto):

```
1      beqz t0, xset  
2      _____  
3      j skip  
4 xset: li t2, 1  
5 skip: nop
```

Quale istruzione manca nel codice del Compilatore B (linea 2) per renderlo logicamente equivalente a quello del Compilatore A?

Scegli un'alternativa:

- ☐ a. and t3, t0, t1
- ☐ b. beq t0, t1, xset
- ☒ c. beqz t1, xset ✓
- ☐ d. or t3, t0, t1
- ☐ e. bnez t1, skip
- ☐ f. bne t0, t1, skip

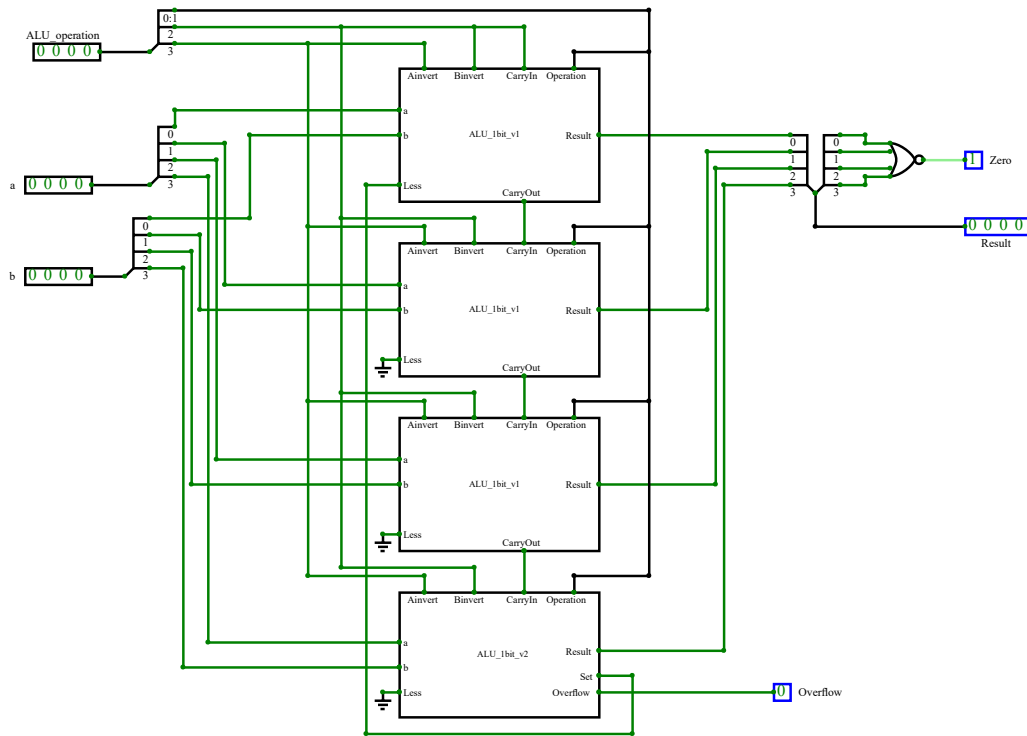
La risposta corretta è: beqz t1, xset

Domanda 7

Risposta non data

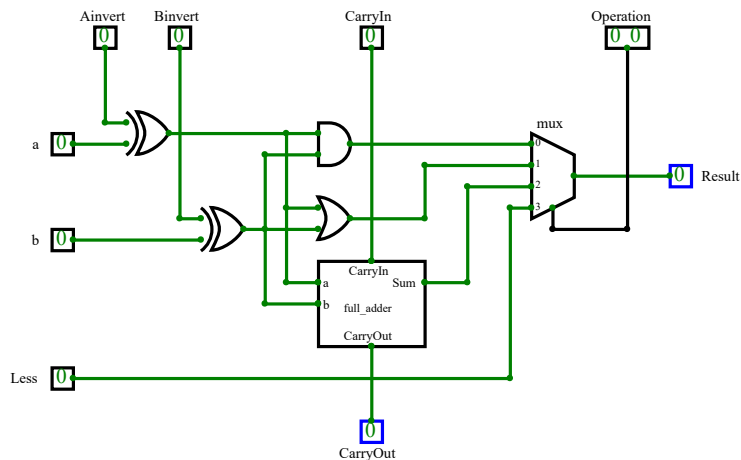
Punteggio max.: 1,00

Si consideri una ALU a 4 bit come realizzata in laboratorio.

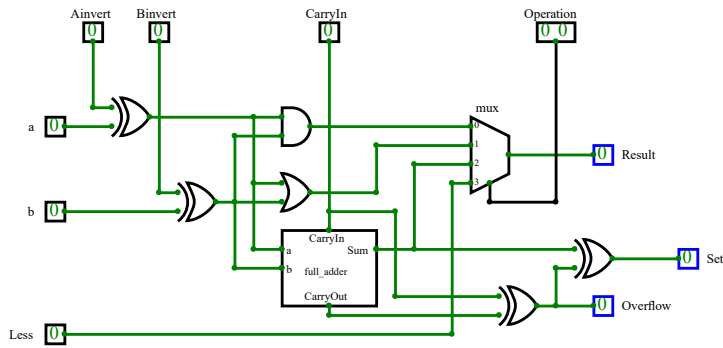


Questa ALU a 4 bit è composta da 3 ALU a 1 bit di "tipo 1" e 1 ALU a 1 bit di "tipo 2", che sono realizzate come nelle figure seguenti:

ALU_1bit_v1



ALU_1bit_v2



Assumendo che siano portati in ingresso i seguenti valori binari:

- a = 0011
- b = 1010
- ALU_operation = 0000

Determinare le uscite dall'ALU.

- (0.50 punti) Result = ✖
- (0.25 punti) Zero = ✖
- (0.25 punti) Overflow = ✖

Scrivere il valore **Result** in uscita dall'ALU come stringa di 4 bit, es. 1010.

Domanda 8

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Assumendo una architettura a 32 bit, supponiamo che i registri **x5** e **x6** contengano i valori **0x00000001** e **0xFFFFFFFF** rispettivamente. Determinare quale sarà il contenuto di **x30** dopo l'esecuzione di un'istruzione che somma i contenuti di **x5** e **x6**.

x30 = 0x00000000 ✔

Il risultato in **x30** è corretto, o si è verificato un overflow? Non si è verificato un overflow ✔

Si supponga nuovamente che i registri **x5** e **x6** contengano i valori riportati sopra (**0x00000001** e **0xFFFFFFFF**). Determinare quale sarà il contenuto di **x30** dopo l'esecuzione di un'istruzione che sottrae il contenuto di **x6** da **x5**.

x30 = 0x00000002 ✔

Il risultato in **x30** è corretto o si è verificato un overflow? Non si è verificato un overflow ✔

Domanda 9

Parzialmente corretta

Punteggio ottenuto 0,50 su 1,00

Determinare se sono vere o false le seguenti affermazioni:

- Il loader fa il collegamento dei moduli oggetto formando un unico modulo eseguibile da caricare in memoria.
-  Il problema delle forward reference è affrontato dal linker, usando la *Symbol Table* come struttura di appoggio. 
- L'assemblatore (assembler) traduce i codici mnemonici nei codici numerici corrispondenti alle istruzioni ISA.
-  La tabella dei codici operativi (opcode table) è usata dall'assemblatore solo nella seconda passata. 

Domanda **10**

Risposta corretta

Punteggio ottenuto 1,00 su 1,00

Considerando un processore RISC-V a 32 bit, si supponga che il registro PC abbia il valore **0x00400008** (in esadecimale), e che l'istruzione corrente sia la prima istruzione riportata nel frammento di codice RISC-V di seguito.

Determinare l'indirizzo che sarà presente nel PC dopo l'esecuzione dell'istruzione **blt t0, zero, cont1**

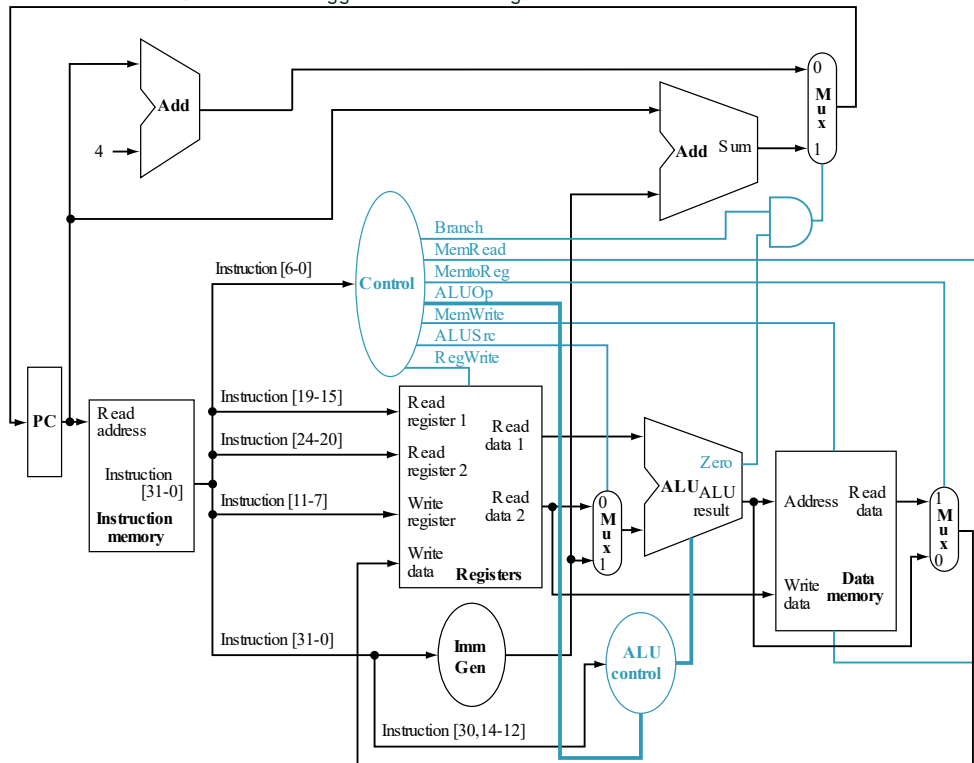
```

addi t0, zero, 4
addi t1, zero, -4
xor t0, t0, t1
blt t0, zero, cont1
addi t0, zero, 0
jal zero, cont0

cont1:
    addi t0, zero, 1

cont0:
    nop
  
```

Come riferimento, il PC viene aggiornato dalla seguente rete nel RISC-V:



Scegli un'alternativa:

- ☐ a. **0x00400014**
- ☒ b. **0x00400020** ✓
- ☐ c. **0x00400010**
- ☐ d. **0x0040001C**
- ☐ e. **0x00400018**

La risposta corretta è: **0x00400020**

Domanda **11**

Risposta corretta

Punteggio ottenuto 4,00 su 4,00

Scrivere una procedura chiamata `countequal(arr1, arr2, size)` che conta quante posizioni hanno valori identici in entrambi gli array. Le variabili `arr1` e `arr2` sono array di `word`, e `size` è il numero di elementi in ciascun array.

La funzione restituisce il numero di posizioni dove `arr1[i] == arr2[i]`.

Il valore di ritorno deve essere lasciato nel registro `a0`.

Il seguente codice in C implementa `countequal` (convertirlo in RISC-V):

```
// int (in C) è equivalente a word (in RISC-V)
int countequal(int arr1[], int arr2[], int size) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (arr1[i] == arr2[i]) {
            count++;
        }
    }
    return count;
}
```

Attenzione:

- Incollare solo la funzione `countequal` (in RISC-V) nel campo sottostante
- Prestare attenzione alle convenzioni di chiamata!
- Utilizzare il seguente codice `main` per lo sviluppo e il debug nel simulatore

```
.globl _start
.data
    arr1: .word 5, 3, 7, 2, 8
    arr2: .word 5, 4, 7, 2, 1
    size: .word 5

.text
_start:
    # call countequal
    la a0, arr1
    la a1, arr2
    la a2, size
    lw a2, 0(a2)
    jal ra, countequal

    #exit
    li a7, 10
    ecall

*****
# completare la funzione countequal nel campo sottostante
```

Risposta (Criteri di penalizzazione: 0 %)

Ripristina risposta

```
1  /*
2  int countequal(int arr1[], int arr2[], int size) {
3      int count = 0;
4      for (int i = 0; i < size; i++) {
5          if (arr1[i] == arr2[i]) {
6              count++;
7          }
8      }
9      return count;
10 }
11 */
```



	Test	Atteso	Ottenuto	
✓	<pre>.section .data arr1: .word 5, 3, 7, 2, 8 arr2: .word 5, 4, 7, 2, 1 size: .word 5</pre>	a0: 3	a0: 3	✓

Tutti i test sono stati superati ✓

► Visualizza/Nascondi soluzione dell'autore (Asm)

Risposta corretta

Punteggio di questo invio: 4,00/4,00.

Commento:

ok

Domanda **12**

Risposta corretta

Punteggio ottenuto 4,00 su 4,00

Scrivere una procedura chiamata `bothcontain(str1, str2, ch)` che riceve due stringhe `str1` e `str2` e un carattere `ch` e restituisce:

- 1 se entrambe le stringhe contengono il carattere `ch`
- 0 altrimenti

La funzione `bothcontain` deve utilizzare la funzione `contains` che riceve una stringa e un carattere e restituisce 1 se la stringa contiene il carattere, 0 altrimenti. Soluzioni che non utilizzano la funzione `contains` saranno considerate errate.

`contains` è già implementata in RISC-V e non deve essere riscritta. Il valore di ritorno deve essere lasciato nel registro `a0`. Il seguente codice in C implementa `bothcontain` (convertirlo in RISC-V):

```
// int (in C) è equivalente a word (in RISC-V)
int bothcontain(char *str1, char *str2, char ch) {
    if (contains(str1, ch) && contains(str2, ch))
        return 1;
    else
        return 0;
}
```

Attenzione:

- Incollare solo la funzione `bothcontain` (in RISC-V) nel campo sottostante
- Prestare attenzione alle convenzioni di chiamata!
- Utilizzare il seguente codice `main` per lo sviluppo e il debug nel simulatore

```

.globl _start
.data
    str1: .string "programming"
    str2: .string "algorithm"
    ch: .byte 'a'

.text
_start:
    # chiama bothcontain
    la    a0, str1
    la    a1, str2
    la    a2, ch
    lbu   a2, 0(a2)
    jal   ra, bothcontain

    li    a7, 10
    ecall

#####
# contains
# a0 -> str
# a1 -> character
# ritorna 1 se str contiene il carattere, 0 altrimenti
#####
contains:
    mv     t0, a0
ct1: lbu   t1, 0(t0)
    beqz   t1, ct3
    beq    t1, a1, ct2
    addi   t0, t0, 1
    j      ct1
ct2: li    a0, 1
    ret
ct3: li    a0, 0
    ret

#####
# completare la funzione bothcontain nel campo sottostante

```

Risposta (Criteri di penalizzazione: 0 %)

Ripristina risposta

```

1 bothcontain:
2 addi sp, sp, -16
3 sw ra, 12(sp)
4 sw s0, 8(sp)
5 sw s1, 4(sp)
6 sw s2, 0(sp)
7 mv s0, a1
8 mv s1, a2
9 mv a1, s1
10 jal ra, contains
11 mv s2, a0
12 mv a0, s0
13 mv a1, s1
14 jal ra, contains
15 beqz a0, rz
16 beqz s2, rz
17 j ro
18 rz:
19 li a0, 0
20 j end
21 ro:
22 li a0, 1
23 j end
24
25 end:
26 lw ra, 12(sp)
27 lw s0, 8(sp)
28 lw s1, 4(sp)

```




	Test	Atteso	Ottenuto	
✓	<pre>.section .data str1: .string "programming" str2: .string "algorithm" ch: .byte 'a'</pre>	a0: 1	a0: 1	✓

Tutti i test sono stati superati ✓

► Visualizza/Nascondi soluzione dell'autore (Asm)

Risposta corretta

Punteggio di questo invio: 4,00/4,00.

Commento:

ok (anche se ultima j end inutile)